

Alexander Batrakov - Selected Project Portfolio

Data Science, Data Analytics, Machine Learning, and Software Engineering

Wuppertal, Germany | batrakovalex.astro@gmail.com | github.com/AlexBatrakov | linkedin.com/in/alexbatrakov97

Portfolio Summary

This portfolio complements my one-page CV with six selected projects focused on data science, data analytics, machine learning, and reproducible computational workflows. The projects show how I work with messy data, analytical SQL, ML evaluation, backend-supported experiments, simulation environments, and scientific software engineering.

How to Read This Portfolio

The projects are intentionally diverse and cover complementary capabilities across analytics, ML, backend workflows, simulation, and scientific software. Use the **Role-Relevant Evidence Snapshot** to map job requirements to evidence, then the **Project Overview** and project cards for concrete implementations.

Role-Relevant Evidence Snapshot

Evidence area	What to look for	Projects
Data quality & SQL analytics	Sanitized pipelines, quality labels, data dictionary, artifact reports, DuckDB/PostgreSQL marts, time-aware joins, rolling windows, and analytical views.	Wearable Analytics
ML experimentation & model reliability	Multi-seed selection, controlled ablations, MLflow/artifact tracking, calibration, error analysis, feature selection, baselines, action masks, and conservative holdouts.	PyTorch Pets Wearable Solo Wargame AI
Backend/API & run orchestration	FastAPI services, database-backed run state, SQLAlchemy/Alembic, Redis/RQ workers, normalized configs, run attempts, metrics/artifacts persistence, retries, Docker/CI paths.	Evalynx PyTorch Pets
Statistical diagnostics	Residual statistics, normalized summaries, convergence metrics, normality checks, white-noise fitting, adaptive grids, priors, and candidate reranking.	GravityToolsNext Wearable
Simulation & AI-agents	Deterministic simulator, staged decision engine, hidden-information rules, legal-action masks, replay traces, policy families, exact/reference workflows, and fixed-seed benchmarks.	Solo Wargame AI
Scientific computing	Julia package APIs, ODE solvers, shooting methods, equation-of-state (EoS) abstractions, sensitivity diagnostics, structured run artifacts, docs, CI, and tests.	GravityToolsNext StructureSolver
Reproducibility & engineering	Tests, docs, CI, MLflow, CLI or Docker reviewer paths, deterministic replay, data contracts, machine-readable artifacts, and documented assumptions.	All projects

Project Overview

Project	Primary signal	Portfolio role
Wearable Analytics	DA / DS / Time Series	Quality-aware Garmin analytics workflow with SQL marts, feature engineering, and leakage-aware modeling.
PyTorch Pets Classifier	ML / Reliability / Deployment	Reproducible PyTorch CV workflow with MLflow tracking, diagnostics, calibration, and FastAPI/Docker deployment.
Evalynx	Backend / Experiment Platform	FastAPI/PostgreSQL/Redis control plane for computational runs, async workers, retries, metrics, and artifacts.
Solo Wargame AI	Simulation / AI Agents / Evaluation	Deterministic hidden-information simulator with legal-action masks, replay, fixed-seed benchmarks, and policy families.
GravityToolsNext.jl	Statistical Diagnostics / RSE	Julia framework for external model-fitting workflows with structured metrics, residual diagnostics, priors, and adaptive parameter search.
StructureSolver.jl	Numerical Computing / Julia	Julia package for ODE simulations, shooting methods, parameter scans, and sensitivity analysis.

Role signals: [Data Analytics](#) [Data Science](#) [Time Series](#) [SQL](#) [ML Evaluation](#)

Summary: Privacy-first Garmin analytics workflow that turns messy JSON/FIT exports into sanitized parquet datasets, quality labels, SQL marts, statistical validation, monitoring features, and leakage-aware next-sleep modeling.

Built

- Implemented a staged local pipeline for Garmin JSON/FIT exports: discovery, ingestion, sanitization, parquet checkpoints, data dictionary, quality labels, and suspicious-day reports.
- Built DuckDB/PostgreSQL analytics layers with views, CTE/window queries, day-to-next-sleep joins, rolling recovery metrics, stress quartiles, risk flags, and modeling-ready SQL extracts.
- Engineered monitoring and sleep-aligned feature tables from minute-level HR/stress records, including semantic wake/sleep windows, coverage/gap quality features, and prior-history/state-context predictors.
- Developed leakage-aware modeling workflows with feature-set comparisons, feature selectors, clipping/calibration options, dummy baselines, mixed random/temporal validation, finalist refits, and fixed-future-holdout diagnostics.

Selected evidence

- 677 daily rows and 1.56M+ minute-level HR/stress observations processed from local Garmin exports.
- 617 / 677 days labeled strict-good; 21 corrupted stress-only artifact days flagged before analysis/modeling.
- DuckDB/PostgreSQL SQL layer demonstrates day-to-next-sleep joins, rolling windows, quartiles, risk flags, and modeling-ready extracts.
- 52,812 linear-family configurations screened; validation-selected Huber model reduced fixed-future-holdout MAE by 15.8% versus dummy median; 150 tests passing locally.

Best used as evidence for: data quality, SQL analytics, analytics engineering, time-series feature engineering, leakage-aware validation, reproducible modeling workflows, and conservative DS/ML evaluation.

Stack: Python, pandas, NumPy, SciPy, scikit-learn, DuckDB, PostgreSQL, parquet, Typer CLI, pytest, GitHub Actions

PyTorch Pets Classifier

Role signals: [Machine Learning](#) [Computer Vision](#) [ML Evaluation](#) [Model Reliability](#) [Deployment](#)

Summary: Reproducible PyTorch computer-vision workflow with ResNet18 transfer learning, controlled ablations, MLflow-backed experiment tracking, three-seed model selection, structured error analysis, calibration checks, and a FastAPI/Docker/Azure deployment path.

Built

- Implemented a config-driven PyTorch/torchvision training and evaluation pipeline for ResNet18 transfer learning on 37 Oxford-IIIT Pets classes, with early stopping, scheduler options, top-k metrics, checkpoint metadata, and checkpoint-aware inference.
- Built a reproducible experiment workflow with YAML configs, isolated run artifacts, MLflow logging, comparison tables, seed-family summaries, public experiment reports, and Makefile commands.
- Added diagnostics beyond accuracy: per-class metrics, top confusion pairs, per-sample predictions, confidence histograms, overconfident errors, error-delta comparison, and calibration checks.
- Packaged the selected checkpoint behind a tested FastAPI inference service with Docker, Azure Container Apps deployment, GitHub Actions CI/CD, a pinned model manifest, checksum verification, and smoke checks.

Selected evidence

- Documented 21 experiment configurations with controlled comparisons across scheduler, resolution, weight decay, batch/LR, freeze, augmentation, and seed-sweep choices.
- Selected the showcase recipe by three-seed robustness rather than a single best run; test top-1 accuracy: 88.29% ± 0.15%.
- Built structured error-analysis and reliability reports covering per-class performance, confusion pairs, confidence behavior, calibration metrics, and before/after error deltas.
- Exposed the model through FastAPI, Docker, Azure Container Apps, GitHub Actions CI/CD, pinned release artifacts, and post-deploy smoke checks.
- Local suite: 62 passing tests across API behavior, prediction, transforms, tracking, calibration, experiment analytics, and error-delta logic.

Best used as evidence for: controlled ML experimentation, MLflow-backed tracking, robustness-aware model selection, model diagnostics, calibration, experiment analytics, PyTorch transfer learning, and model-to-API deployment.

Stack: Python, PyTorch, torchvision, ResNet18, MLflow, FastAPI, Docker, Azure Container Apps, GitHub Actions, pytest, Makefile

Role signals: [Backend](#) [API](#) [Async Jobs](#) [Experiment Platform](#) [Software Engineering](#)

Summary: FastAPI/PostgreSQL/Redis backend control plane for reproducible computational runs, turning ad-hoc scripts and scattered logs into API-driven submission, database-backed lifecycle state, async execution, retry history, and structured result persistence.

Built

- Implemented a FastAPI backend with project/run endpoints, service/repository/worker layers, SQLAlchemy/Alembic persistence, and PostgreSQL-backed run state.
- Modeled durable execution history with `Project`, `Run`, and `RunAttempt` records, including normalized configs, config hashes, lifecycle status, metrics, artifacts, warnings/errors, and timestamps.
- Added Redis/RQ asynchronous execution with worker-side attempt processing, retry-safe failed-run handling, stale-attempt replay protection, and per-attempt artifact directories.
- Integrated external computation through bounded runner adapters and a subprocess JSON contract; packaged a reviewer stack with Docker Compose and GitHub Actions CI smoke tests.

Selected evidence

- 8 API endpoints covering health, project management, run submission, run listing/detail, project-scoped runs, and failed-run retry.
- SQLAlchemy/Alembic model with `Project`, `Run`, and `RunAttempt`, plus migrations for result fields and attempt backfill.
- Redis/RQ execution path with retry semantics that preserve failed attempts and protect current run snapshots from stale workers.
- Docker Compose stack with api, worker, postgres, redis, and migrate services; public reviewer can run the stub demo locally.
- GitHub Actions validates pytest, module compilation, Docker stack build, migrations, service startup, and HTTP smoke tests; 29 pytest tests.

Best used as evidence for: backend API design, relational state modeling, async job orchestration, retry/failure semantics, reproducible experiment execution, Docker Compose packaging, and CI-backed software engineering workflow.

Stack: Python, FastAPI, SQLAlchemy, Alembic, PostgreSQL, Redis, RQ, Pydantic, Docker Compose, GitHub Actions, pytest

Solo Wargame AI

Role signals: [Simulation](#) [AI Agents](#) [ML Evaluation](#) [Research Software](#) [Software Engineering](#)

Summary: Deterministic Python simulation and policy-evaluation stack for stochastic hidden-information decisions, turning rulebook-style logic into testable state/action contracts, replayable episodes, fixed-seed benchmarks, and agent-evaluation workflows.

Built

- Translated a prose-heavy stochastic rule system into explicit Python domain contracts: state, actions, staged decision contexts, legal-action generation, transitions, observations, terminal outcomes, and validation.
- Implemented deterministic RNG snapshots, replay traces, strict TOML mission parsing/validation, hidden-information rules, Gym-style environment wrappers, fixed action catalogs, and policy-facing legality masks.
- Built evaluation surfaces for random, heuristic, rollout-search, exact-guided, and PyTorch masked actor-critic agents, with fixed benchmark seeds and separated train/model-selection/evaluation seed roles.
- Added reproducible tooling around the simulator: CLI commands, JSON episode batch runner, SQLite exact/policy-audit artifacts, machine-readable metrics, local browser play, pytest/Ruff verification, and CI.

Selected evidence

- Layered Python package with deterministic simulator, mission configs, legal-action engine, replay, Gym-style wrappers, agents, CLIs, local browser play, and tests.
- Fixed-seed Mission 1 benchmark surface: random 11/200, learned best 144/200, heuristic 157/200, rollout-search 195/200; exact fair ceiling about 0.95.
- Learned masked actor-critic remains explicitly included as one of the policy families, with baseline gaps surfaced rather than hidden.
- Machine-readable JSON evaluation workflows include request/response schemas, execution metadata, aggregate metrics, and optional JSONL episode rows.
- Verification surface: 356 passing tests and clean Ruff check.

Best used as evidence for: simulation engineering, AI-agent evaluation, deterministic testing, legal-action masking, baseline-controlled experiments, replay/debugging, and reproducible evaluation workflows.

Stack: Python, PyTorch, Gym-style wrappers, TOML, JSON/JSONL, SQLite, pytest, Ruff, CLI tooling, stdlib HTTP server

Role signals: [Statistical Diagnostics](#) [Experiment Orchestration](#) [Scientific Computing](#) [Research Software](#) [Julia](#)

Summary: Julia framework for reproducible external model-fitting workflows, turning run artifacts into structured metrics, residual diagnostics, white-noise fits, prior-aware analyses, adaptive grids, and tested documentation/Docker surfaces.

Built

- Designed typed Julia settings and deterministic job materialization for external TEMPO/TEMPO2 runs, including validated paths, staged inputs/outputs, workspace policies, output capture, cleanup rules, and structured artifacts.
- Implemented parsers and result objects for Tempo2-style stdout, par/tim files, residual files, parameter estimates, internal iterations, convergence summaries, fit metrics, and failure states.
- Built statistical diagnostic layers for residual statistics, normalized summaries, backend grouping, convergence metrics, normality tests, and backend-aware white-noise EFAC/EQUAD/offset fitting.
- Developed prior-aware and adaptive parameter-search workflows with analytic/grid/sampled priors, posterior-weight-style aggregation, JLD2 persistence, and pure-Julia adaptive 2D grids for expensive objective surfaces.

Selected evidence

- Converts raw external-run artifacts into structured result objects with chi-square, weighted RMS, normalized residual summaries, convergence status, parameter estimates, and pre/post fit deltas.
- Implements residual and white-noise diagnostic modules with backend-aware fit objects, EFAC/EQUAD/offset concepts, normality diagnostics, and Anderson-Darling-style objective support.
- Combines prior-marginalized execution and adaptive-grid search for parameter exploration over expensive model-fitting surfaces.
- Public repository includes Documenter docs, examples, CI workflows, Docker development baseline, and a broad Julia test suite.

Best used as evidence for: statistical diagnostics, residual analysis, uncertainty-aware model evaluation, adaptive parameter search, external model-fitting orchestration, structured metric extraction, and reproducible Julia workflows.

Stack: Julia, TEMPO/TEMPO2, HypothesisTests, Distributions, KernelDensity, Optim, StatsBase, JLD2, Documenter.jl, Docker

StructureSolver.jl

Role signals: [Scientific Computing](#) [Numerical Methods](#) [Sensitivity Analysis](#) [Research Software](#) [Julia](#)

Summary: Julia package for ODE-based simulation workflows with equation-of-state (EoS) abstractions, shooting methods, parameter scans, ForwardDiff sensitivity diagnostics, tests, and docs.

Built

- Designed a modular Julia package separating EoS backends, physical models, solver regimes, simulation drivers, utilities, documentation, and tests.
- Implemented direct and shooting simulation regimes for ODE-based solves, including integration controls, boundary-condition targets, structured quantity extraction, and named outputs.
- Built scan and sensitivity workflows for single solves, family/grid scans, N-D shooting scans, ForwardDiff-based derivative extraction, and NLSolve-based boundary matching.

Selected evidence

- Implements reusable EoS, model, regime, simulation, shooting, scan, and sensitivity-analysis interfaces rather than one-off numerical scripts.
- Converts domain inputs into documented named outputs and diagnostics such as masses/radii, boundary residuals, central quantities, sound speed, and derivative outputs.
- Public Julia package with Documenter docs, examples, CI workflows, release/citation metadata, and local verification: 69 / 69 tests passed.

Best used as evidence for: numerical scientific computing, Julia package design, ODE simulation workflows, shooting methods, parameter scans, sensitivity analysis, and documented output contracts.

Stack: Julia, DifferentialEquations.jl, ForwardDiff.jl, NLSolve.jl, Interpolations.jl, Documenter.jl, GitHub Actions

Common Working Style

Across the projects, I emphasize explicit data contracts, reproducible runs, baseline comparisons, statistical diagnostics, testable components, documented assumptions, and inspectable artifacts. The common pattern is to make inputs visible, preserve intermediate outputs, compare against baselines, and keep workflows reviewable.